НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №5. Управління подіями. Контрольовані та неконтрольовані компоненти

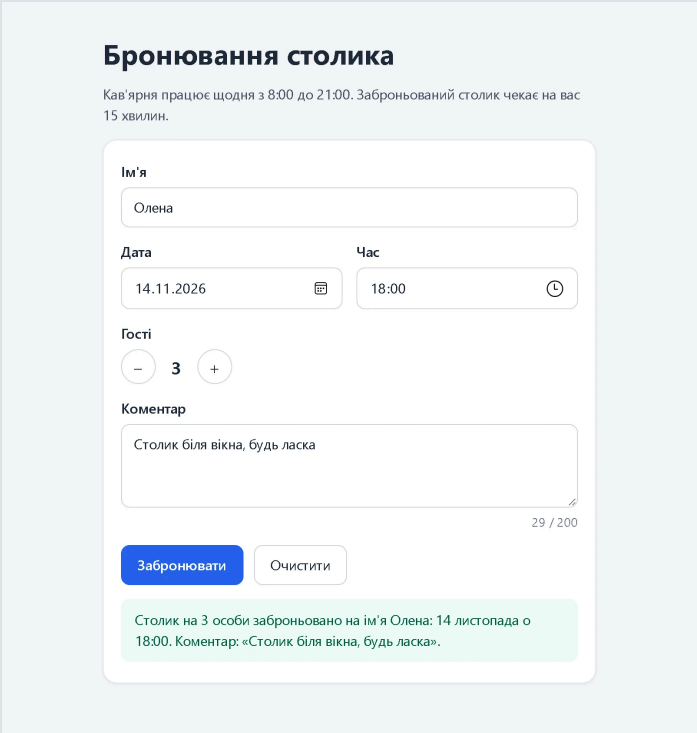
з дисципліни «Розробка вебзастосунків на React»

> **Примітка щодо актуальності.** Документ підготовлено станом на 8 листопада 2026 року: React 19, Vite 8, ESLint 10 з `eslint-plugin-react-hooks` 7, Prettier 3, React DevTools 8. У разі відмінностей орієнтуйтеся на документацію React (§2).

1. Мета роботи

Навчитися обробляти події форми, клавіатури й миші в React і розрізняти контрольовані та неконтрольовані поля й компоненти: зібрати форму бронювання столика, у якій частину значень зберігає браузер, а частину — стан React, і для кожного значення вирішити, кому воно має належати.

Очікуваний результат: у теці `lab-5/` репозиторію `react-labs` — форма «Бронювання столика», яка надсилається без перезавантаження сторінки й показує підтвердження; лічильник під коментарем змінюється під час введення; `Ctrl+Enter` надсилає форму з будь-якого поля; кнопки − і + змінюють кількість гостей від 1 до 8 і не надсилають форму; «Очистити» очищає всю форму й ставить курсор у поле «Ім'я»; у `lab-5/README.md` заповнено таблицю контрольованих і неконтрольованих полів; застосунок відкривається на Vercel.



2. Корисні посилання

Теоретичної частини в цій роботі немає. Перед заняттям прочитайте сторінки за посиланнями 1 і 2. Решту відкривайте, коли до них відсилає завдання.

1. Реагування на події — головна сторінка роботи: обробники подій, передавання їх пропсами, поширення (спливання) подій, `preventDefault`.
2. Контрольовані та неконтрольовані компоненти — розділ сторінки «Поширення стану між компонентами»: коли компонент називають контрольованим (§4.5–4.6).
3. `<input>` — довідка про поля: читання значень під час надсилання форми (`FormData`), контрольоване поле, типові помилки (англійською, §4.2–4.3).
4. Форми і Неконтрольовані компоненти — те саме українською, у старій документації React. Читайте їх заради ідеї: приклади коду там на класових компонентах.
5. Manipulating the DOM with Refs — `useRef` і фокус на полі (англійською, §4.6).
6. FormData — об'єкт із даними форми (MDN, англійською, §4.2).
7. `<button>`: атрибут `type` — чому кнопка у формі за замовчуванням надсилає її (MDN, англійською, §4.1, §4.5).
8. KeyboardEvent.key, requestSubmit() і Event.currentTarget — натиснута клавіша, надсилання форми з коду й елемент з обробником (MDN, англійською, §4.4).
9. Synchronizing with Effects: Subscribing to events і beforeunload — підписка на подію вікна, функція очищення й попередження перед виходом зі сторінки (англійською, §4.8).

3. Завдання лабораторної роботи

1. Додати в репозиторій `react-labs` теку `lab-5/` зі стартового проєкту й подивитися, як браузер надсилає форму без JavaScript (§4.1).
2. Надсилати форму без перезавантаження сторінки: `onSubmit`, `preventDefault`, `FormData` (§4.2).
3. Зробити коментар контрольованим полем із лічильником символів (§4.3).
4. Надсилати форму клавішами `Ctrl+Enter` (§4.4).
5. Оживити кнопки у компоненті `GuestsStepper` (§4.5).
6. Навчити кнопку «Очистити» очищати всю форму (§4.6).
7. Перевірки, GitHub і Vercel (§4.7), звіт (§5).

Усі завдання §4.1–4.7 виконуються на занятті (орієнтовно 20–30 хвилин); звіт (§5) можна надіслати після нього. Робота оцінюється в 1 бал. §4.8 — додаткове завдання для тих, хто впорався раніше; воно не оцінюється.

AI. Код цієї роботи пишіть самостійно. AI-автодоповнення в редакторі можна залишити ввімкненим, але кожен запропонований рядок ви маєте вміти пояснити на захисті. Доручати роботу AI-агентові не рекомендовано: мета — навчитися самому вирішувати, кому належить кожне значення, браузеру чи React, і як подія його змінює.

4. Хід виконання роботи

У ЛР-4 поле пошуку було контрольованим: його значення жило в стані, а onChange цей стан змінював. Тепер у вас форма з п'ятьма полями, і для кожного ви вирішите, чи потрібен йому стан. Після кожного кроку перевіряйте сторінку в браузері, консоль і вкладку **Components** ⚛. Файли зберігайте як у ЛР-1 (`Ctrl+S`): Prettier відформатує їх під час збереження.

4.1. Стартовий проєкт

1. Відкрийте в редакторі теку `react-labs` і перевірте, що `git status` чистий. Якщо працюєте на іншому комп'ютері, спершу клонуйте `react-labs` з GitHub.
2. Завантажте lab-5.zip і розпакуйте його в провіднику (Windows: **Extract All**, macOS: подвійний клік). Вийде тека `lab-5`. Перемістіть її в корінь `react-labs`, поруч із теками попередніх робіт.
   Не розпаковуйте архів командою в терміналі в корені `react-labs`: файли в ньому лежать без теки й замінили б кореневі `package.json` і `.gitignore`.
3. У корені репозиторію виконайте `git status -u`: усі нові файли мають бути в `lab-5/`.

4. Встановіть залежності в корені й запустіть dev-сервер:

   ```
   npm install
   cd lab-5
   npm run dev
   ```

   Відкрийте http://localhost:5173: на сторінці форма бронювання столика. Dev-сервер не зупиняйте до §4.7, а для команд git відкрийте другий термінал (у VS Code — **+** на панелі терміналу).
5. Спробуйте форму, поки в ній немає жодного обробника подій:
   1. Натисніть «Забронювати» з порожніми полями: браузер попросить заповнити ім'я. Так працює атрибут `required`: поля «Ім'я», «Дата» й «Час» обов'язкові. Валідацію форм ви розберете в ЛР-6.
   2. Заповніть ім'я, дату й час і натисніть «Забронювати»: сторінка перезавантажилася, поля очистилися, а в адресному рядку з'явилося `?name=…&date=…&time=…&guests=2&comment=`. Так браузер надсилає форму без JavaScript: збирає значення полів з атрибутом `name` і відкриває сторінку з ними в адресі.
   3. Знову заповніть поля й натисніть «+»: сталося те саме. Кнопка всередині форми без атрибута `type` — це кнопка надсилання (`type="submit"`). Ви виправите це в §4.5.
6. Що вже є в проєкті:

Файл	Що в ньому
`src/App.jsx`	компонент `App`: заголовок, короткий опис і `BookingForm`
`src/components/BookingForm.jsx`	форма: поля, кнопки «Забронювати» й «Очистити», порожній абзац для підтвердження
`src/components/GuestsStepper.jsx`	кількість гостей: кнопки − і +, число й приховане поле `guests`
`src/describeBooking.js`	функція `describeBooking(booking)`: складає з даних форми текст підтвердження
`src/index.css`	усі стилі сторінки
`README.md`	таблиця контрольованих і неконтрольованих полів, яку ви заповните в §4.3–4.6
`package.json`, `eslint.config.js`, `.prettierrc.json`	скрипти й налаштування ESLint і Prettier, як у ЛР-1
7. Зробіть коміт у корені репозиторію: так до нього потрапить і `package-lock.json`.

   ```
   git add .
   git commit -m "chore: add lab 5 starter"
   ```

Далі шляхи до файлів (`src/components/BookingForm.jsx`) — відносно `lab-5/`.

4.2. Надсилання форми без перезавантаження

Щоб форма надсилалася без перезавантаження, React має обробити подію `submit` і скасувати те, що браузер робить за замовчуванням.

1. У `src/components/BookingForm.jsx` імпортуйте `useState` і функцію `describeBooking`:

   ```
   import { useState } from 'react'
   import { describeBooking } from '../describeBooking.js'
   ```

2. На початку функції `BookingForm`, перед `return`, оголосіть стан для даних останнього бронювання й обробник події:

   ```
   const [booking, setBooking] = useState(null)

   function handleSubmit(e) {
     e.preventDefault()
     const formData = new FormData(e.currentTarget)
     setBooking(Object.fromEntries(formData))
   }
   ```

   - `e` — об'єкт події: React передає його обробнику першим аргументом;
   - `e.preventDefault()` скасовує стандартну поведінку браузера: надсилання форми з перезавантаженням сторінки;
   - `e.currentTarget` — елемент, якому передано обробник, тобто `<form>`. `new FormData(форма)` збирає значення всіх її полів з атрибутом `name`;
   - `Object.fromEntries(formData)` робить із них звичайний об'єкт: `{ name: 'Олена', date: '2026-11-14', … }`.

3. Передайте обробник формі:

   ```
   <form className="booking-form" onSubmit={handleSubmit}>
   ```

   Передавайте саму функцію, а не результат її виклику: з `onSubmit={handleSubmit()}` React викликав би її під час рендеру, і ESLint позначить це помилкою `react-hooks/set-state-in-render`.

4. Виведіть підтвердження в останній абзац форми:

   ```
   <p role="status" className="status">
     {booking && describeBooking(booking)}
   </p>
   ```

   Поки `booking` дорівнює `null`, абзац порожній. Атрибут `role="status"` просить програми зчитування з екрана оголосити текст, щойно він з'явиться.

Що має вийти:

- після «Забронювати» сторінка не перезавантажується й адреса не змінюється, а під кнопками з'являється «Столик на 2 особи заброньовано на ім'я …»;
- введені значення залишаються в полях;
- у **Components** ⚛ у `BookingForm` один хук — `State`: `null` до надсилання й об'єкт із даними форми після нього. Поки ви вводите ім'я, жоден хук не змінюється: значення поля зберігає браузер, а React читає його лише раз, під час надсилання. Таке поле називають **неконтрольованим** (uncontrolled);
- «+» досі надсилає форму: сторінка вже не перезавантажується, але підтвердження оновлюється. Це ви виправите в §4.5.

Коміт: `feat: submit the booking without a page reload`. **Довідка:** Додавання обробників подій, Запобігання стандартній поведінці, Reading the input values when submitting a form, FormData.

4.3. Контрольоване поле: коментар із лічильником

Під коментарем стоїть лічильник «0 / 200». Щоб він змінювався під час введення, React має знати текст коментаря під час кожного рендеру. Тому коментар стане **контрольованим** полем, як поле пошуку в ЛР-4: його значення зберігається в стані, а onChange цей стан змінює.

1. Під станом booking оголосіть ще один:

   ```
   const [comment, setComment] = useState('')
   ```

2. Додайте до `<textarea>` два атрибути:

   ```
   value={comment}
   onChange={(e) => setComment(e.target.value)}
   ```

3. У лічильнику замініть `0` на довжину коментаря:

   ```
   <p id="comment-counter" className="counter">
     {comment.length} / 200
   </p>
   ```

   Довжина коментаря — не окремий стан: її обчислюють із `comment` під час кожного рендеру.

4. Увімкніть підсвічування рендерів: у вкладці **Components** ⚛ натисніть шестерню ⚙ праворуч від поля пошуку компонентів, на вкладці **General** позначте **Highlight updates when components render** і закрийте налаштування. Тепер DevTools обводить рамкою кожен компонент, який рендериться. Введіть кілька літер у поле «Ім'я», потім — у «Коментар». Заповніть рядки «Ім'я» й «Коментар» таблиці в `README.md`.

Що має вийти:

- лічильник змінюється з кожною літерою, а після 200 символів поле більше нічого не приймає (атрибут `maxLength`);
- у Components ⚛ у `BookingForm` два хуки `State`, і другий змінюється з кожною літерою коментаря;
- коментар потрапляє в підтвердження: `FormData` читає й контрольовані поля;
- у таблиці заповнено два рядки.

Коміт: `feat: count the comment characters` (разом із таблицею). **Довідка:** Controlling an input with a state variable, Контрольовані та неконтрольовані компоненти, Форми.

4.4. Клавіатура: `Ctrl+Enter` надсилає форму

У коментарі `Enter` переносить рядок, тож надсилати форму звідти будемо клавішами `Ctrl+Enter` (на macOS — ⌘+Enter), як у багатьох полях для повідомлень.

1. Додайте в `BookingForm` обробник:

```
function handleKeyDown(e) {
  if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) {
    e.currentTarget.requestSubmit()
  }
}
```

2. Передайте його **формі**, а не полю коментаря:

```
<form
  className="booking-form"
  onSubmit={handleSubmit}
  onKeyDown={handleKeyDown}
>
```

   - `e.key` — назва натиснутої клавіші; `e.ctrlKey` і `e.metaKey` кажуть, чи затиснуто `Ctrl` або ⌘;
   - клавішу натискають у полі, а обробник стоїть на `<form>`. Подія `keydown` спливає від поля до всіх його батьківських елементів, тож один обробник на формі спрацьовує для кожного поля. `e.target` — поле, у якому натиснули клавішу, а `e.currentTarget` — форма, якій передано обробник;
   - `requestSubmit()` надсилає форму так само, як кнопка «Забронювати»: браузер перевіряє обов'язкові поля, а потім запускає подію `submit`, яку обробляє ваш `handleSubmit`.

Що має вийти:

- `Ctrl+Enter` у коментарі надсилає форму, і підтвердження оновлюється;
- якщо ім'я порожнє, браузер просить його заповнити, як і після натискання кнопки;

- `Enter` без `Ctrl` у коментарі переносить рядок, а в полі «Ім'я» надсилає форму: так браузер поводиться з однорядковими полями й без вашого коду.

Коміт: `feat: submit the form with Ctrl+Enter`. **Довідка:** Поширення події, KeyboardEvent.key, requestSubmit(), Event.currentTarget.

4.5. Миша: кнопки − і +

Кількість гостей показує `GuestsStepper`. Поки число в ньому не змінюється, а кнопки надсилають форму. Тепер `GuestsStepper` зберігатиме кількість гостей у власному стані, а приховане поле `<input type="hidden" name="guests">` передаватиме її у `FormData` під час надсилання. Що нового в коді з кроку 1:

- `type="button"` робить кнопку звичайною: вона більше не надсилає форму (§4.1);
- `onClick={handleIncrement}` передає функцію, яку React викличе після кліку. З `onClick={handleIncrement()}` вона виконалася б під час рендеру, і ESLint позначить це помилкою `react-hooks/set-state-in-render`;
- `setGuests((g) => g + 1)` — функція оновлення: React передає в неї актуальне значення стану. `Math.min` і `Math.max` не дають вийти за межі від 1 до 8;
- на межі кнопка отримує `aria-disabled`: вона виглядає неактивною, а програма зчитування з екрана називає її недоступною. Атрибут `disabled` тут не підходить: кнопка, яка стає `disabled`, втрачає фокус, і користувач клавіатури втрачає місце на сторінці;

- `GuestsStepper` сам зберігає своє значення, а батьківський компонент задає лише початкове — `defaultValue`, як у неконтрольованого `<input defaultValue="…">`. Такий компонент називають **неконтрольованим**.

1. `src/components/GuestsStepper.jsx` має стати таким:

```
import { useState } from 'react'

const MIN = 1
const MAX = 8

export default function GuestsStepper({ defaultValue }) {
  const [guests, setGuests] = useState(defaultValue)

  function handleDecrement() {
    setGuests((g) => Math.max(MIN, g - 1))
  }

  function handleIncrement() {
    setGuests((g) => Math.min(MAX, g + 1))
  }

  return (
    <fieldset className="stepper">
      <legend>Гості</legend>
      <div className="stepper-controls">
        <button
          type="button"
          aria-label="Менше гостей"
          aria-disabled={guests === MIN}
          onClick={handleDecrement}
        >
          –
        </button>
        <output>{guests}</output>
        <button
          type="button"
          aria-label="Більше гостей"
          aria-disabled={guests === MAX}
          onClick={handleIncrement}
        >
          +
        </button>
      </div>
      <input type="hidden" name="guests" value={guests} />
    </fieldset>
  )
}
```

2. У `BookingForm.jsx` передайте початкове значення:

```
<GuestsStepper defaultValue={2} />
```

3. Із підсвічуванням рендерів (§4.3) клацніть − і + кілька разів і заповніть рядок «Гості (§4.5)» таблиці.

Що має вийти:

- − і + змінюють число від 1 до 8 і не надсилають форму;
- на 1 кнопка − виглядає неактивною, на 8 — кнопка +;
- перейдіть на «+» клавішею Tab і натисніть `Enter` чи `Пробіл`: число змінюється, як після кліку мишею, а фокус лишається на кнопці навіть на 8;
- у підтвердженні правильна форма слова: «на 1 особу», «на 3 особи», «на 5 осіб»;
- у Components ⚛ у `GuestsStepper` хук `State` і проп `defaultValue: 2`.

Коміт: `feat: choose the number of guests` (разом із таблицею). **Довідка:** Додавання обробників подій, Іменування пропсів обробників подій, `<button>`: атрибут `type`, Контрольовані та неконтрольовані компоненти.

4.6. «Очистити»: хто володіє значенням

1. Заповніть усі поля, виберіть 5 гостей, напишіть коментар і натисніть «Очистити». Кнопка з `type="reset"` без жодного JavaScript повертає кожне поле форми до початкового значення. Запишіть, які поля очистилися, а які — ні.
2. **Обробник скидання.** Додайте в `BookingForm`:

```
function handleReset() {
  setBooking(null)
  setComment('')
}
```

   і передайте його формі поруч з іншими: `onReset={handleReset}`. Подія `reset` настає, коли форму скидають кнопкою «Очистити».
3. **Контрольований `GuestsStepper`.** Кількість гостей так не очистити: вона зберігається в стані `GuestsStepper`, а `BookingForm` до стану іншого компонента доступу не має. Підніміть стан у `BookingForm`, як у ЛР-4:
   - у `BookingForm` оголосіть стан і передайте його `GuestsStepper`, а в `handleReset` додайте `setGuests(2)`:

     ```
     const [guests, setGuests] = useState(2)
     ```

     ```
     <GuestsStepper value={guests} onChange={setGuests} />
     ```

   - у `GuestsStepper` приберіть `useState` разом з імпортом, отримуйте пропси `value` і `onChange` замість `defaultValue`, у JSX замініть `guests` на `value`, а обробники

кнопок змініть так:

```
function handleDecrement() {
  onChange(Math.max(MIN, value - 1))
}

function handleIncrement() {
  onChange(Math.min(MAX, value + 1))
}
```

Тепер значення задає батьківський компонент, а `GuestsStepper` лише повідомляє про зміни через onChange — з тими самими пропсами, що й контрольований `<input>`. Такий компонент називають **контрольованим**.

4. **Фокус.** Після очищення курсор має опинитися в полі «Ім'я». Щоб звернутися до DOM-елемента, потрібен реф. Імпортуйте `useRef` разом з `useState` і оголосіть реф поруч зі станом:

```
import { useRef, useState } from 'react'
```

```
const nameRef = useRef(null)
```

Додайте полю «Ім'я» атрибут `ref={nameRef}`, а в кінець `handleReset` — рядок `nameRef.current.focus()`. Після рендеру React записує в `nameRef.current` сам елемент `<input>`. Читайте реф в обробниках подій, а не під час рендеру: інакше ESLint позначить помилку `react-hooks/refs`.

5. Заповніть рядок «Гості (§4.6)» таблиці.

Що має вийти:

- «Очистити» очищає всі поля, лічильник показує «0 / 200», гостей знову 2, підтвердження зникає, а курсор стоїть у полі «Ім'я»;
- у Components ⚛ у `GuestsStepper` немає хуків, зате є пропси `value` і onChange; у `BookingForm` — три `State` і `Ref`;
- таблицю в `README.md` заповнено повністю.

Коміт: `feat: clear every field with the reset button` (разом із таблицею).
Довідка: Контрольовані та неконтрольовані компоненти, Example: Focusing a text input, Неконтрольовані компоненти.

4.7. Перевірки, GitHub і Vercel

1. Зупиніть dev-сервер (`Ctrl+C`) і виконайте в `lab-5/`:

```
npm run lint
npm run format:check
npm run build
```

Усі три команди мають пройти без помилок, а `npm run lint` — ще й без попереджень (`warning`). Якщо `format:check` назве файл, відкрийте його, збережіть ще раз і закомітьте: `style: format lab 5 files`.

2. Відправте коміти на GitHub: `git push`. Історія комітів роботи:
    1. `chore: add lab 5 starter`;
    2. `feat: submit the booking without a page reload`;
    3. `feat: count the comment characters`;
    4. `feat: submit the form with Ctrl+Enter`;
    5. `feat: choose the number of guests`;
    6. `feat: clear every field with the reset button`.
3. На vercel.com → **Add New** → **Project** ще раз імпортуйте `react-labs`: це буде ще один проєкт з того самого репозиторію. Перед **Deploy** налаштуйте:

Поле	Значення
Project Name	`react-lab-5`
Root Directory	**Edit** → `lab-5`
Framework Preset	Vite (Vercel розпізнає його сам)
Output Directory	`dist`

4. Після деплою скопіюйте адресу застосунку з вкладки **Overview** проєкту, поле **Domains**, відкрийте її й надішліть форму ще раз.

На цьому роботу завершено.

4.8. Додаткове завдання

Не обов'язкове й не оцінюється.

«Незбережені зміни». Якщо користувач заповнив форму й випадково перезавантажує чи закриває сторінку, браузер може його попередити.

1. Оголосіть у `BookingForm` стан `isDirty` (`useState(false)`) і передайте формі ще один обробник:

    ```
    onInput={() => setIsDirty(true)}
    ```

    Подія `input` настає в полі з кожною зміною й спливає до форми, тож один обробник стежить за всіма полями: і контрольованими, і неконтрольованими. У `handleSubmit` і `handleReset` додайте `setIsDirty(false)`.
2. Додайте ефект, який підписується на подію `beforeunload` вікна, лише поки у формі

є незбережені зміни (`useEffect` імпортуйте разом з іншими хуками):

```
useEffect(() => {
  if (!isDirty) return

  function handleBeforeUnload(e) {
    e.preventDefault()
  }

  window.addEventListener('beforeunload', handleBeforeUnload)
  return () => window.removeEventListener('beforeunload',
handleBeforeUnload)
}, [isDirty])
```

Функція, яку повертає ефект, — функція очищення: React викликає її перед наступним запуском ефекту й тоді, коли компонент зникає зі сторінки.

3. Введіть ім'я й натисніть F5: браузер запитає, чи покинути сторінку. Надішліть форму й натисніть F5 ще раз — запитання немає. Тепер тимчасово приберіть рядок `return () => …` і повторіть: після надсилання браузер усе одно питає. Поверніть рядок.
4. Змініть лише кількість гостей і натисніть F5. Чи попереджає браузер? Чому?

Коміт: `feat: warn before leaving with unsaved changes`.

5. Вимоги до звіту

Тема листа:

```
[React] ЛР-5: події, контрольовані й неконтрольовані компоненти
```

У листі вкажіть теку лабораторної на GitHub і адресу застосунку на Vercel:

```
GitHub repository: https://github.com/<username>/react-labs/tree/main/lab-5
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. знімок екрана сторінки після бронювання з відкритою вкладкою **Components** ⚛, де вибрано `BookingForm` і видно його хуки;
2. знімок екрана Components ⚛, де вибрано `GuestsStepper` і видно його пропси `value` і `onChange`;
3. знімок екрана `npm run lint`, `npm run format:check` і `npm run build` без помилок;
4. чи виконано §4.8;
5. чи використовували AI-агента і для чого;
6. короткий висновок: що зроблено й що було складним.

Таблиця контрольованих і неконтрольованих полів лежить у `lab-5/README.md`, у звіт її можна продублювати.

6. Усний захист

Формат здачі передбачає усну співбесіду за результатами роботи. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного коду. Викладач може попросити показати відповідь у коді чи DevTools або змінити код наживо. Питання з групи «Незбережені зміни» ставлять тим, хто виконав §4.8.

Питання для підготовки:

Обробники подій

- Що таке обробник події в React? Чим `onClick={handleClick}` відрізняється від `onClick={handleClick()}`? Що повідомляє ESLint у другому випадку?
- Чому обробники називають `handle...`, а пропси з обробниками — `on...`? Наведіть приклади з вашого коду.
- Коли обробник зручніше оголосити окремою функцією, а коли — стрілковою функцією просто в JSX?
- Що таке об'єкт події? Які його властивості ви використали в цій роботі?
- Чи можна змінювати стан в обробнику події? А під час рендеру?

Стандартна поведінка браузера

- Що робить браузер, коли форму надсилають без JavaScript? Звідки в адресі взялися `name=…&date=…`?
- Що робить `e.preventDefault()`? Наведіть інші приклади стандартної поведінки, яку скасовують так само.
- Який `type` має кнопка всередині форми, якщо атрибут не вказано? Що змінилося, коли ви додали `type="button"`?
- Чим `form.requestSubmit()` відрізняється від `form.submit()`?
- Чому `Enter` у полі «Ім'я» надсилає форму, хоч ви цього не програмували?

Спливання подій

- Що таке спливання (bubbling) подій? Чому обробник `onKeyDown` на `<form>` спрацьовує, коли клавішу натискають у полі?
- Чим `e.target` відрізняється від `e.currentTarget`? Що з них ви використали в `handleSubmit` і `handleKeyDown` і чому?
- Як зупинити спливання події? Коли це може знадобитися?
- Що таке фаза перехоплення (capture) і як обробити подію в ній у React?

Клавіатура й миша

- Як дізнатися, яку клавішу натиснуто й чи затиснуто `Ctrl`? Навіщо перевіряти ще й `metaKey`?
- Чому `onClick` на кнопці спрацьовує й тоді, коли її натискають клавішею `Enter` чи `Пробіл`? Чи спрацював би він так само на `<div>`?
- Чим `aria-disabled` відрізняється від `disabled`? Чому кнопки − і + не використовують `disabled`?
- Навіщо кнопкам − і + атрибут `aria-label`?

Контрольовані й неконтрольовані поля

- Що таке контрольоване поле? А неконтрольоване? Наведіть по прикладу з вашої форми.
- Де зберігається значення поля «Ім'я»? Як `BookingForm` дізнається його?
- Чому коментар має бути контрольованим, а ім'я — ні? Чи можна показати лічильник символів для неконтрольованого поля?
- Що показало підсвічування рендерів під час введення імені й коментаря? Чому?
- Чим `value` відрізняється від `defaultValue`?
- Що означає попередження «A component is changing an uncontrolled input to be controlled»? Як його отримати й як виправити?
- Що станеться, якщо дати полю `value` без `onChange`?
- Як прочитати значення неконтрольованого поля без `FormData`?

Контрольовані й неконтрольовані компоненти

- Коли компонент називають контрольованим, а коли — неконтрольованим?
- Яким був `GuestsStepper` у §4.5 і яким став у §4.6? Що змусило його змінити?
- Чим `<GuestsStepper value={guests} onChange={setGuests} />` схожий на `<input value={…} onChange={…} />`?
- Як підняття стану з ЛР-4 пов'язане з перетворенням неконтрольованого компонента на контрольований?
- Навіщо в `GuestsStepper` приховане поле `<input type="hidden">`?
- Чому в §4.5 `setGuests` отримує функцію `(g) => …`, а в §4.6 `onChange` отримує готове число?

Скидання форми й рефи

- Які поля очистила кнопка «Очистити» без обробника, а які ні? Чому?
- Коли настає подія `reset`? Що робить ваш `handleReset`?
- Що таке реф? Чим `useRef` відрізняється від `useState`?
- Коли React записує елемент у `nameRef.current`? Чому реф не читають під час рендеру?
- Навіщо після очищення переводити фокус у поле «Ім'я»?

Доступність форми

- Навіщо полям `<label>`? Як ще можна пов'язати підпис із полем?
- Навіщо `<fieldset>` і `<legend>` у лічильнику гостей?
- Навіщо абзацу з підтвердженням `role="status"`? Чому він є на сторінці, навіть коли порожній?
- Навіщо полю коментаря `aria-describedby`?

Незбережені зміни

- Чому один обробник `onInput` на формі бачить зміни в усіх полях?
- Чому кнопки − і + не позначають форму зміненою?
- Що сталося без функції очищення в ефекті з `beforeunload`? Чому?

7. Критерії готовності

Робота оцінюється в 1 бал після захисту (§6).

Критерій	Мінімальна вимога
Проєкт	`lab-5/` у репозиторії `react-labs`; у теці `npm run lint` проходить без помилок і попереджень, `format:check` і `build` — без помилок
Надсилання	сторінка не перезавантажується; дані читаються через `FormData`; підтвердження показує ім'я, дату, час, кількість гостей і коментар
Неконтрольовані поля	ім'я, дата й час — без `value` і без стану
Коментар	контрольоване поле; лічильник обчислюється з довжини коментаря, а не зберігається окремо
Клавіатура	`Ctrl+Enter` (⌘+Enter) надсилає форму з будь-якого поля; обробник — на `<form>`
Гості	кнопки з `type="button"` не надсилають форму; значення від 1 до 8; `GuestsStepper` — контрольований компонент із пропсами `value` і `onChange`
Очищення	«Очистити» очищає всі поля, лічильник, кількість гостей і підтвердження; курсор переходить у поле «Ім'я»
Таблиця	таблицю в `README.md` заповнено
GitHub	коміти з §4.7
На занятті	усі коміти §4.1–4.7 відправлено на занятті (час `git push` видно в **Activity** репозиторію)
Vercel	проєкт з Root Directory `lab-5`; адреса показує сторінку з останнього коміту `main`
Звіт	усі пункти §5